

David Nguyen

CSEN 296B-2 — AI-Driven Software Development

Lab 1.2: Architecture Decision Log

Chosen language / technical path:

Next.js 14 (App Router) with TypeScript, Supabase for auth/database/realtime, and Tailwind + shadcn/ui on the frontend. I've used the App Router before on a side project so I'm not starting from scratch, and Supabase bundles auth + Postgres + realtime into one managed service which keeps the number of things I have to wire together low for a solo quarter project.

Weaker prompt:

"Design a simple storage system for AstraNotes so users can save notes."

This gave me a very shallow answer. It suggested a single notes table with id, title, body, and created_at columns, a basic Express CRUD API, and that was about it. No mention of who owns the data, no failure handling, no privacy boundaries, nothing about how the design stays testable or maintainable. It basically gave me the first thing that would compile without thinking about what could go wrong. If I shipped that to real users I'd have no access control and no way to reason about edge cases.

Stronger prompt:

"Act as a Senior Lead Engineer. I am defining the architecture for AstraNotes, a secure and modular note-taking system built with Next.js 14 (App Router) and Supabase (Postgres, Auth, Realtime). This is a solo quarter project with a 10-week timeline. Design the storage architecture. Think step by step. Show the main components, their responsibilities, the data model, failure handling, privacy boundaries, and how the design remains testable and maintainable over time. Then compare this against an alternative where the backend is a separate Express API with Firebase, and explain the trade-offs."

Excerpt from the stronger response:

"Step 1 — Core Components and Responsibilities: The storage layer has three bounded areas: (1) a Notes Service responsible for CRUD operations and version snapshots, (2) an Access Control layer enforced via Supabase Row-Level Security policies at the database level rather than in application code, and (3) a Sync Service that manages real-time collaboration channels. Separating these responsibilities means you can test access control independently of note content logic.

Step 2 — Data Model: The schema uses four tables: notes (id, owner_id, folder_id, title, body_tsvector, updated_at), folders (id, owner_id, name), note_shares (note_id, shared_with, permission_level), and note_versions (id, note_id, content_snapshot, created_at). The owner_id column is the trust boundary—every RLS policy filters on it.

Step 3 — Failure Handling: If the Realtime channel drops mid-edit, the client queues changes locally and reconciles on reconnect. If a write to the versions table fails, the note save itself should still succeed—version history is a secondary concern and should not block the primary write path.

Step 4 — Trade-off vs. Express + Firebase: Firebase gives you real-time sync with less setup, but you lose Postgres-level RLS and full-text search. You'd need to scatter authorization checks across Express middleware, making the trust boundary harder to audit. For a solo dev who needs privacy enforcement to be verifiable in one place, Supabase RLS is the more defensible choice."

Verdict:

Going with the Supabase architecture. The stronger prompt forced the AI to reason step by step through components, failure modes, and privacy boundaries instead of jumping straight to a schema. The thing that sold me was the failure handling breakdown—the weaker response didn't mention what happens when something goes wrong at all, while the stronger one explicitly separated the primary write path from version history so a secondary failure doesn't block the user. That's the kind of design decision I'd actually want to defend in a review.